

INTERNSPARK VLSI DESIGN INTERNSHIP

Task 3 Report: FSM-Based Vending Machine Controller Design, Synthesis & Static Timing Analysis (STA)

Submitted By: Mudarakola Devi priya

July 2026

1 Introduction & Objectives

The objective of this task is to implement a Finite State Machine (FSM) based Vending Machine controller in structural Verilog, execute functional verification via a pure Verilog testbench utilizing Verilator and GTKWave, perform logic synthesis using Yosys to target the OSU 0.18 μ m standard cell library, and conduct Static Timing Analysis (STA) via OpenSTA to evaluate setup and hold closure.

The design flow begins with a high-level behavioral FSM description, transitions to an automated gate-level compilation environment, and concludes with structural path constraint checks. This end-to-end framework highlights the critical relationship between structural logic implementation and physical propagation delay metrics.

2 Behavioral Specification & State Machine Flow

The controller governs a vending machine that dispenses an item worth 15 cents. It processes two single-bit input lines capturing local currency inputs: a Nickel (5 ϕ) and a Dime (10 ϕ).

To guarantee hazard-free transitions, the design specifies 5 distinct operation tracking points:

- S_0 (0 ϕ baseline code)
- S_5 (5 ϕ accumulated balance)
- S_10 (10 ϕ accumulated balance)
- S_15 (15 ϕ balance achieved – dispense item with 0 ϕ change)
- S_20 (20 ϕ balance achieved – dispense item with 5 ϕ change)

3 Toolchain Pipeline Execution Steps

3.1 Step 1: RTL Modeling & Verification Cleanup

The core state machine logic and its accompanying stimulus testbench are written in pure Verilog text files. A sanitation filter command (`sed -i`) is run directly in the Linux terminal to strip hidden Windows carriage return formatting characters, preventing compilation errors.

3.2 Step 2: Automated Functional Simulation & Waveform Probing

An automated shell script launches Verilator with native timing flags enabled to compile the behavioral Verilog files. The simulation executable generates a Value Change Dump (`.vcd`) file capturing all signal transitions. GTKWave is opened to visually inspect the waveforms.

3.3 Step 3: Logic Synthesis & Technology Mapping

The verified behavioral RTL is loaded into the Yosys synthesis engine alongside the standard cell library. Yosys runs optimization passes to translate behavioral logic into a structural gate-level netlist. Abstract conditional logic blocks are replaced with real physical cell primitives (such as AND, NAND, NOR, INV, and DFFSR sequential elements).

3.4 Step 4: Timing Constraints Formulation

A Synopsys Design Constraints (.sdc) file is created to model the physical operating environment of the chip. This file establishes a baseline clock target with a 10ns period, specifies input/output delay windows at the boundaries, and defines the structural pin loads.

3.5 Step 5: Static Timing Analysis Probing

The OpenSTA engine is launched interactively in the terminal to perform post-synthesis timing sign-off. The cell library, synthesized gate netlist, and SDC constraints are read into the tool sequentially to calculate final system slack parameters.

4 Verification Logs & Timing Evaluation Reports

4.1 A. Gate-Level Netlist Mapping Analysis

The structural output from Yosys reveals that the compiler automatically optimized the FSM logic using a One-Hot Encoding scheme. The compiler allocates a 5-bit register vector where exactly one bit stays active per operating cycle, which significantly reduces the depth of combinational next-state decoding gates.

4.2 B. Static Timing Verification Insights

4.2.1 1. Hold Timing Verification (Minimum Delay Path)

The execution log for the minimum delay path analysis indicates excellent hold margins:

- **Data Arrival Time:** 0.34 ns
- **Data Required Time:** 0.01 ns
- **Slack Status:** MET (+0.34 ns)

The internal storage registers provide sufficient short-path delays, ensuring that data remains completely stable right after the active clock edge to prevent latching hazards.

4.2.2 2. Setup Timing Verification (Maximum Delay Path)

Evaluating the maximum delay path under the initial timing boundaries flags a critical path constraint violation:

- **Data Arrival Time:** 14.13 ns

- **Data Required Time:** 9.88 ns
- **Slack Status:** VIOLATED (-4.25 ns)

The propagation delay through the combinational path takes 14.13 ns. This length exceeds the total allowable 10 ns clock cycle budget minus required library setup margins.